Variables in Terraform

Introduction

Imagine you are deploying **multiple servers** using Terraform.

- **How do you avoid writing the same values repeatedly?**
- **How do you make your Terraform configurations dynamic and reusable?**
- **How do you pass values like instance type, region, or database name easily?**

Terraform Variables solve these problems!

Section 1: Learn

What is a Variable in Terraform?

A variable in Terraform is a placeholder that allows users to pass dynamic values into configurations.

Variables help to:

1. **Reuse Terraform configurations** without hardcoding values.
2. **Easily modify infrastructure settings** without editing multiple files.
3. **Make Terraform files cleaner and more manageable.**

Why Do We Need Variables?

Without Variables	**With Variables**
Hardcoded values in Terraform files	Flexible configurations
Difficult to update settings	Easily modify values

Without Variables	With Variables
Prone to human errors	Ensures consistency
No reusability	Reusable across different environments

Types of Variables in Terraform

1. **String Variable** – Stores text values.
2. **Number Variable** – Stores numeric values.
3. **Boolean Variable** – Stores true or false.
4. **List Variable** – Stores multiple values in a list.
5. **Map Variable** – Stores key-value pairs.

An Interesting Anecdote

A cloud engineer at a startup used **Terraform without variables** and had to **manually update 50+ configuration files** every time an instance type changed. After switching to **Terraform variables**, they **saved hours of work by updating just one variable!**

Section 2: Practice

Step-by-Step Guide to Using Terraform Variables

1. Defining a Simple Variable

Create a file variables.tf:

```
variable "instance_type" {
 description = "Type of EC2 instance"
 type        = string
 default     = "t2.micro"
```

```
}
```

Modify main.tf to use this variable:

```
resource "aws_instance" "web_server" {
  ami           = "ami-12345678"
  instance_type = var.instance_type
}
```

Run Terraform commands:

```
terraform init
terraform apply
```

What happens here?

- Terraform **automatically takes the default value** of "t2.micro".

---

2. Overriding Default Variable Values

Instead of using the default value, you can provide a value at runtime:

```
terraform apply -var="instance_type=t3.micro"
```

Why is this useful?

- Allows **changing values dynamically** without modifying Terraform files.

---

3. Using a List Variable

Modify variables.tf:

```
variable "instance_types" {
  description = "List of instance types"
  type        = list(string)
```

```
  default     = ["t2.micro", "t3.micro", "t3.small"]
}
```

Use this in main.tf:

```
resource "aws_instance" "web_server" {
  ami           = "ami-12345678"
  instance_type = var.instance_types[0]  # Uses "t2.micro"
}
```

Run Terraform:

```
terraform apply
```

Why is this useful?

- **Allows easy switching between instance types**.

---

4. Using a Map Variable

Modify variables.tf:

```
variable "instance_map" {
  description = "Map of instance types by environment"
  type        = map(string)
  default = {
    dev  = "t2.micro"
    prod = "t3.medium"
  }
}
```

Use this in main.tf:

```
resource "aws_instance" "web_server" {
  ami           = "ami-12345678"
  instance_type = var.instance_map["dev"]  # Uses "t2.micro"
}
```

Run Terraform:

```
terraform apply
```

Why is this useful?

- **Dynamically selects values based on environment (dev/prod).**

---

5. Using Environment Variables to Pass Values

Instead of using -var, you can use environment variables:

```
export TF_VAR_instance_type="t3.large"
terraform apply
```

Why is this important?

- **Enhances security** by keeping values out of code.

Real-World Example

A **cloud engineering team** needs to:

1. **Deploy multiple environments (dev, test, prod).**
2. **Ensure all configurations remain flexible.**
3. **Easily switch between different cloud providers.**

What did they do?

1. Used **Terraform variables** to define cloud settings.
2. Applied variables **to dynamically configure environments**.
3. Automated deployments using **CI/CD pipelines**.

Result? The team **saved time, reduced errors, and improved scalability!**

Section 3: Know More

Frequently Asked Questions

1. **What is the purpose of variables in Terraform?**
   - Variables **make Terraform configurations reusable and flexible**.
2. **Can I pass multiple variables in Terraform?**
   - Yes, use -var multiple times:

```
terraform apply -var="instance_type=t3.micro" -var="region=us-east-1"
```

3. **What happens if I don't provide a variable value?**
   - Terraform **uses the default value** from variables.tf.
4. **How do I define variables in a separate file?**
   - Create a terraform.tfvars file:

```
instance_type = "t3.micro"
```

5. Then apply:

```
terraform apply -var-file="terraform.tfvars"
```

6. **Where can I practice Terraform variables?**
   - Use **AWS Free Tier** and experiment with Terraform configurations.

Conclusion

Terraform **Variables simplify configurations, improve reusability, and enhance flexibility**.

Start by **defining simple variables, using lists and maps, and passing values dynamically**, and soon you'll be **writing scalable Terraform configurations!**